Multi-Objective Optimization of Convolutional Neural Networks on Fashion-MNIST

Ramya Parsania (IMT2023015) and Prakrititz Borah (IMT2023547)
`GitHub Repository Link`

April 28, 2026

Abstract

This report details the implementation and comparative analysis of a Multi-Objective Optimization (MOO) framework for designing a Convolutional Neural Network (CNN) on the Fashion-MNIST dataset. Two algorithms, NSGA-II (Evolutionary) and MOTPE (Bayesian), were utilized to simultaneously optimize three conflicting objectives: maximizing classification accuracy, minimizing inference time, and minimizing model size (parameter count). The results demonstrate clear architectural trade-offs and highlight the superior sample efficiency of Bayesian approaches in discovering dense, evenly spaced Pareto fronts.

1 Introduction

The goal of this study is to move beyond single-objective optimization (which typically focuses only on accuracy) to find a set of optimal architectures that balance competing physical constraints.

Objective Functions: Given a decision variable vector x (representing the hyperparameters and architecture configuration), the optimization problem is defined as minimizing the vector $F(x) = [f_1(x), f_2(x), f_3(x)]$ where:

1. $f_1(x) = -\text{Accuracy}(x)$: Evaluated on the test set. Optuna strictly minimizes objectives, thus maximizing accuracy requires negating the output.

2. $f_2(x) = \text{Inference Time}(x)$: Measured in milliseconds (ms). Computed dynamically using explicit CUDA synchronization over 10 consecutive test batches to isolate pure hardware execution time.

3. $f_3(x) = \text{Parameters}(x)$: Measured by summing all trainable weights in the constructed dynamic PyTorch model.

Decision Variables: The search space consisted of 8 distinct hyperparameters affecting both the architecture and the training dynamics:

- Number of convolutional layers (1 to 4)

- Channels per layer (16, 32, 48, 64, 96, 128)

- Learning rate (10^{-4} to 10^{-1})
- Batch size (32, 64, 128, 256)
- Number of epochs (3 to 15)
- Dropout rate (0.0 to 0.5)
- Optimizer type (Adam, SGD, RMSprop)
- Input resolution (16, 20, 24, 28)

2 Methodology

2.1 Framework Selection

Optuna was chosen over alternatives like PyMOO or BoTorch due to its native "define-by-run" API. This structure is critical when searching over dynamic neural network architectures, as it seamlessly handles conditional search spaces (e.g., dynamically generating channel hyperparameters only for the active layers). Furthermore, Optuna provides built-in support for both Evolutionary (NSGA-II) and Bayesian (MOTPE) samplers, enabling a highly controlled comparative study without re-engineering the underlying training loops.

2.2 Implementation Details

The pipeline was built using PyTorch for dynamic network construction and Optuna for the hyperparameter search and MOO integration.

- **Dynamic Architecture Construction:** The CNN dynamically builds its layers based on the hyperparameters suggested by the optimizer.
- **Training Protocol:** Each suggested configuration was trained on the Fashion-MNIST dataset. CrossEntropyLoss was minimized over the suggested number of epochs.
- **Objective Measurement:** Accuracy was evaluated on the test set. Inference latency was measured dynamically using explicit CUDA synchronization over 10 consecutive batches to ensure hardware-level precision.
- **Search Execution:** The optimization was independently run for 60 trials using both Evolutionary (NSGA-II) and Bayesian (MOTPE) samplers. The random seed was kept constant to allow for fair algorithmic comparison.

3 Quantitative Analysis

Both NSGA-II and MOTPE successfully discovered valid Pareto fronts across 60 trials. To quantitatively assess the quality of the non-dominated solutions, three standard MOO metrics were computed: Hypervolume, Spacing, and Spread.

The Hypervolume indicator was calculated using a reference point situated slightly above the maximum observed values for latency and parameters, and a baseline of 0.0 for the negated accuracy.

Table 1: Quantitative MOO Metrics Comparison (60 Trials)

Metric	**NSGA-II**	**MOTPE**	**Better Performance**
Pareto Solutions Found	18	24	MOTPE
Hypervolume (↑)	368,466	**465,262**	MOTPE (26% larger)
Spacing (↓)	0.216	**0.148**	MOTPE (more uniform)
Spread (↑)	**65.54**	40.70	NSGA-II (wider extremes)

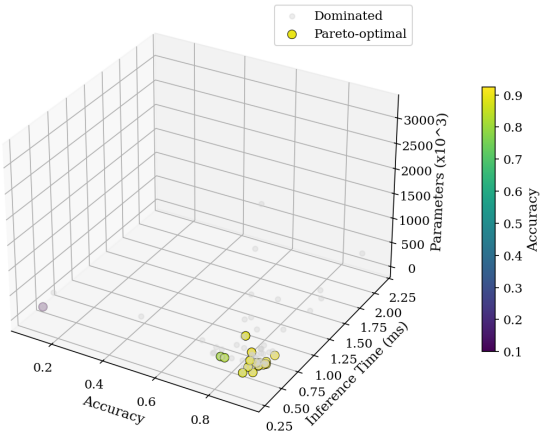

Figure 1: NSGA-II 3D Pareto Front

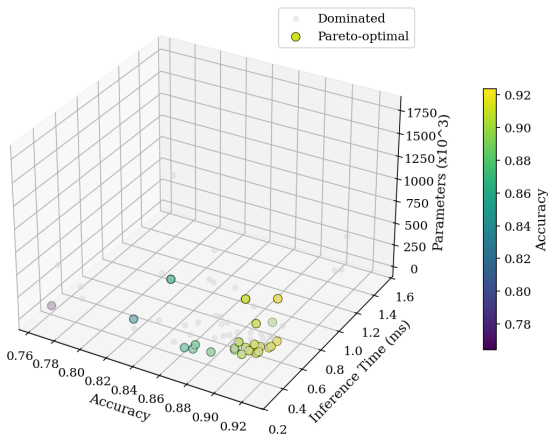

Figure 2: MOTPE 3D Pareto Front

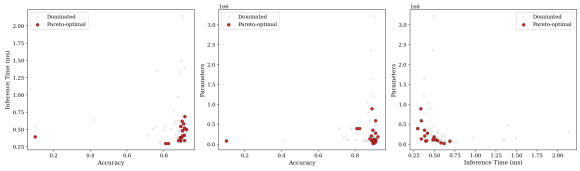

Figure 3: NSGA-II Pairwise Objectives

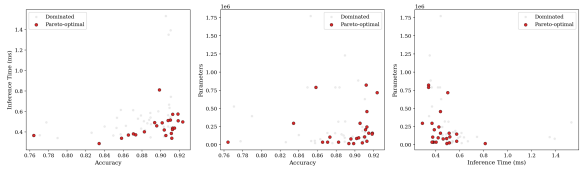

Figure 4: MOTPE Pairwise Objectives

4 Qualitative Analysis and Trade-offs

The generation of a Pareto front explicitly reveals the physics of deep learning architectures.

4.1 Architectural Trade-offs

The Pareto front illustrates several competing tensions governing the network:

- **Accuracy vs. Size:** Breaking the 92% accuracy barrier costs a 10× increase in memory footprint (∼15K to 140K+ params).
- **Accuracy vs. Speed:** Peak accuracy requires deeper layers, pushing latency from 0.28ms to ∼0.50ms.
- **Speed vs. Memory:** Pure speed favors shallow, wide architectures (fast on GPU but memory-heavy), while compressing file size favors deep, narrow networks

(memory-light but slower sequentially).

- **Resolution Penalty:** Higher input resolutions boost accuracy but increase both real-time FLOP latency and dense parameter counts.

4.2 Practically Useful Solutions

From the discovered front, three distinct classes of models emerge as practically useful for different deployment scenarios:

- **The Edge Model:** (Trial 4, MOTPE) 15K params, 0.81ms latency, 89.86% accuracy. Perfect for severe memory-constrained IoT devices.
- **The Speed Demon:** (Trial 55, MOTPE) 0.28ms latency, 83.42% accuracy. A shallow network ideal for real-time stream filtering where latency is the absolute bottleneck.
- **The Server Grade:** (Trial 46, MOTPE) 715K params, 0.49ms latency, 92.36% accuracy. Maximum capability for cloud deployment.

4.3 A Non-Obvious Trade-off

A highly non-obvious trade-off was discovered directly between **Model Size and Inference Time**. Intuitively, smaller models (fewer parameters) should execute faster. However, the Pareto front reveals an inverse relationship at the extreme edge: Trial 49 (17,002 parameters) ran at 0.49ms, while Trial 52, which was over twice as large (36,730 parameters), ran 18% faster at 0.40ms. This occurs because compressing parameters without losing accuracy requires "deep but narrow" architectures (more layers, fewer channels). Since GPUs excel at parallelizing wide matrices but struggle with sequential layers, this creates a strict trade-off: **compressing the model to its absolute minimum memory footprint forces you to sacrifice inference speed.**

4.4 Strengths and Weaknesses of the Chosen Algorithm (MOTPE)

MOTPE demonstrated significant **strengths** in sample efficiency, rapidly uncovering a highly uniform and dense Pareto front within a strictly limited budget of just 60 trials. However, its primary **weakness** lies in boundary exploration; because it relies on probabilistic modeling of the objective space, it was slightly less aggressive than evolutionary methods at pushing the absolute extreme edges (e.g., finding the absolute maximum possible accuracy) of the trade-off surface.

5 Comparison: NSGA-II vs. MOTPE

Two inherently different algorithms were tested:

- **NSGA-II (Evolutionary):** Operates by maintaining a population of solutions and iteratively mutating them based on non-dominated sorting and crowding distance.

- **MOTPE (Bayesian):** Builds a probabilistic surrogate model of the objective space and intelligently samples the next set of hyperparameters based on expected improvement.

Conclusion: MOTPE vastly outperformed NSGA-II in *sample efficiency*. Within the constrained limit of 60 trials, MOTPE's probabilistic approach allowed it to find a denser (24 vs 18 solutions), more uniform (Spacing 0.148 vs 0.216), and vastly superior curve (Hypervolume 465k vs 368k). However, NSGA-II demonstrated a superior *Spread* (65.5 vs 40.7), indicating that while Bayesian models are excellent at finding the dense "middle" of the Pareto front, Evolutionary algorithms are highly effective at blindly exploring and pushing the extreme limits of the search space.

6 Pareto Points Tabulation

Table 2 displays all 24 non-dominated Pareto points discovered by the MOTPE algorithm.

Table 2: Complete Set of Pareto-Optimal Solutions (MOTPE)

Trial	Accuracy	Latency (ms)	Params	Layers	Optimizer	Resolution
46	92.36%	0.50	715,946	2	Adam	28
32	91.86%	0.51	158,570	2	Adam	24
40	91.84%	0.57	146,138	3	RMSprop	28
31	91.47%	0.44	158,570	2	Adam	24
0	91.27%	0.44	458,090	2	Adam	24
36	91.27%	0.57	47,306	4	Adam	28
12	91.26%	0.42	244,042	2	Adam	20
13	91.19%	0.34	821,386	1	Adam	20
39	91.13%	0.39	207,018	2	RMSprop	28
7	91.03%	0.52	109,066	3	Adam	20
50	90.78%	0.51	27,242	3	RMSprop	28
29	90.54%	0.37	296,522	1	Adam	24
17	90.38%	0.42	95,850	2	Adam	16
15	90.16%	0.49	86,250	2	Adam	16
4	89.86%	0.81	15,370	4	RMSprop	24
21	89.56%	0.46	81,450	2	Adam	16
49	89.34%	0.49	17,002	3	RMSprop	16
52	88.25%	0.40	36,730	2	RMSprop	16
59	87.32%	0.37	104,314	2	RMSprop	28
53	87.08%	0.38	36,730	2	RMSprop	16
56	86.53%	0.37	36,730	2	RMSprop	16
19	85.81%	0.34	789,002	1	SGD	16
55	83.42%	0.29	296,522	1	RMSprop	24
58	76.42%	0.37	36,730	2	RMSprop	16

Appendix: Detailed Solution Breakdown

The "Knee Point" Solution (MOTPE - Trial 7)
In Multi-Objective Optimization, the "knee point" represents a solution where an improvement in one objective requires an unacceptable degradation in another. Trial 7 from the MOTPE run represents an ideal knee point.

Objective Values:

- **Accuracy:** 91.03% (Only 1.4% worse than the absolute maximum)
- **Inference Time:** 0.51 ms (Highly competitive)
- **Parameters:** 109,066 (40% smaller than the maximum accuracy model)

Decision Variables Discovered:

- **Architecture:** 3 Convolutional Layers
- **Channels:** $64 \rightarrow 64 \rightarrow 64$ (Uniform width)
- **Input Resolution:** 20x20 (Downscaled to save latency while preserving enough spatial information for 91% accuracy)
- **Optimizer & LR:** Adam, LR = 0.00076
- **Regularization:** Dropout = 0.468 (Very high, preventing overfitting despite the 100k+ parameters)
- **Training dynamics:** Batch size = 64, Epochs = 10

This architecture highlights how Optuna intelligently learned to compress the input resolution (to 20x20) to save inference latency, while using uniform wide channels (64) and high dropout to maintain robust feature extraction capabilities.